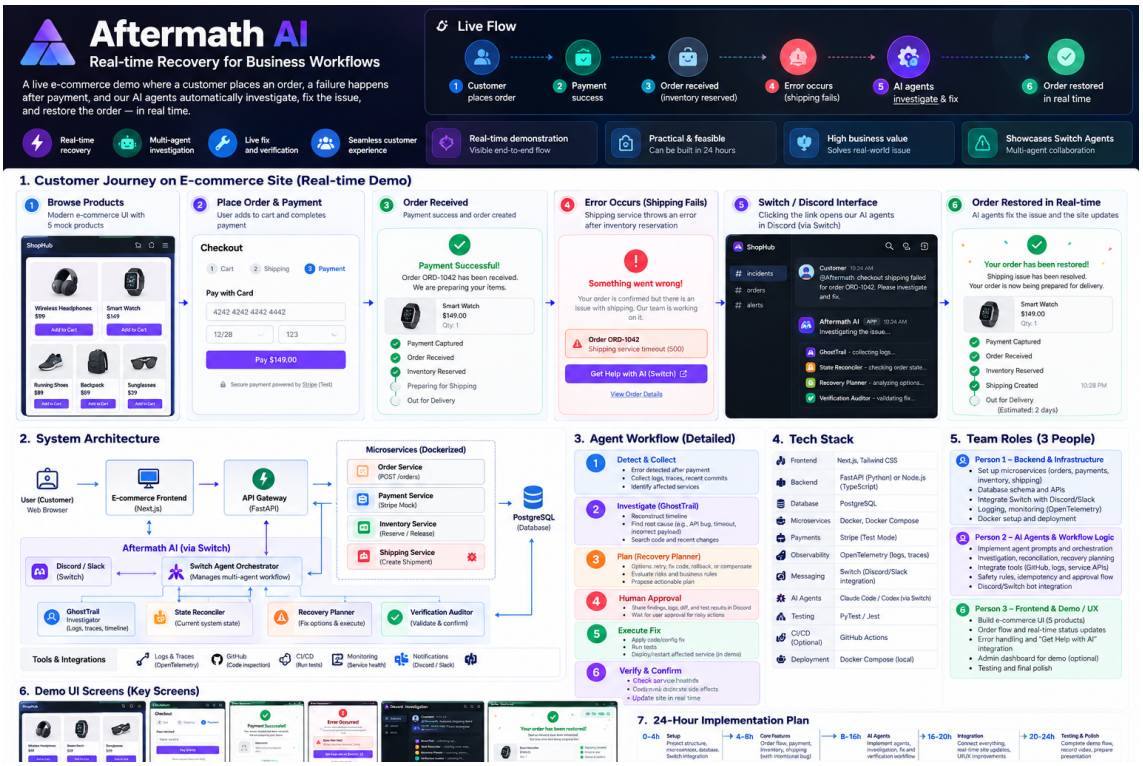
AFTERMATH AI

Real-Time E-Commerce Recovery | Project Workflow & System Design



Aftermath AI is a Switch-connected, multi-agent recovery workflow embedded into a mock e-commerce journey. A customer browses five products, places an order, and completes a simulated payment. If a post-payment service fails (for example, shipping creation times out after the order is received and inventory is reserved), the site shows a clear recovery state and a button that opens the Aftermath AI bot in Discord through Switch. The agents inspect logs and code, reproduce the issue in a controlled environment, propose a fix, run tests, request approval for consequential actions, and verify the resulting business state. The website updates from the backend in near real time and tells the customer the order has been restored only after verification succeeds.

Core promise: A customer sees a real post-payment failure, clicks into the Switch-connected AI team, and returns to the order page after the backend verifies that recovery succeeded.

Hackathon MVP • Five mock products • One seeded post-payment failure • Discord via Switch • No real payment processing

1. Is the Idea Feasible?

Yes—as a tightly scoped demo. The strongest version is not an AI that freely edits and deploys production code. It is a controlled demonstration with one reproducible bug, mock business services, a sandboxed repair/test step, explicit approval, and backend verification. That scope is realistic for three people in a 24-hour sprint if integration begins early.

Recommended demo scenario

Use five mock products. A customer buys one product; simulated payment succeeds, the order is recorded, and inventory is reserved. Then the shipping service hits a seeded bug (for example, an incorrect payload field causes a 500 response). The order is received but fulfillment is pending. After the agent team inspects the relevant code, reproduces the failure, applies a small approved patch in a sandbox, runs tests, and retries safely, the site updates to “Order restored” only after read-back verification.

Important UX and feasibility adjustment

The site should not depend on the customer manually debugging the app. The recovery button opens the incident in the Switch-connected Discord bot for the demo team/operator. The customer order page remains subscribed to backend status updates. In a real product, recovery would be initiated by the platform and the customer would see status updates; the Discord link is an operator/demo entry point, not a requirement that customers join Discord.

2. Customer Journey and End-to-End Workflow

1. Browse

The customer opens the storefront and sees five mock products with images, prices, product details, cart controls, and a clear checkout flow.

2. Checkout and simulated payment

The customer places an order and completes a test-mode/simulated payment. No real card or payment is processed in the hackathon demo.

3. Order received

The order service records the order, the payment mock marks it captured, and inventory is reserved. The page shows a confirmed order with fulfillment still pending.

4. Injected post-payment failure

The shipping service intentionally returns a controlled timeout/500 after the earlier steps succeed. The order is not falsely shown as shipped. Backend records a correlation ID and incident ID.

5. Customer opens AI support

The order page explains that payment and order receipt are confirmed but fulfillment needs attention. The “Resolve with Aftermath AI” button opens the configured Discord/Switch incident room or bot entry point.

6. Collect evidence

The orchestrator fetches relevant structured logs, events, traces, current service states, recent commits, and test results. It never sends secrets or raw payment credentials to an LLM.

7. Investigate and inspect code

The investigator builds a timeline. A code-inspection agent searches the demo repository for the faulty shipping path and identifies a likely cause. Any code execution happens only in a sandbox.

8. Reproduce and propose fix

The test agent creates or runs a regression test against the seeded failure. The fixer proposes a small patch or configuration correction and runs the relevant tests.

9. Plan and approve

The recovery planner checks business invariants, idempotency, and current state. The bot posts the proposed fix/action and evidence. A designated teammate approves consequential actions.

10. Execute safely

The controlled runner applies the fix in the demo environment, restarts/redeploys the mock service if applicable, and retries the business operation with an idempotency key after checking current state.

11. Verify business state

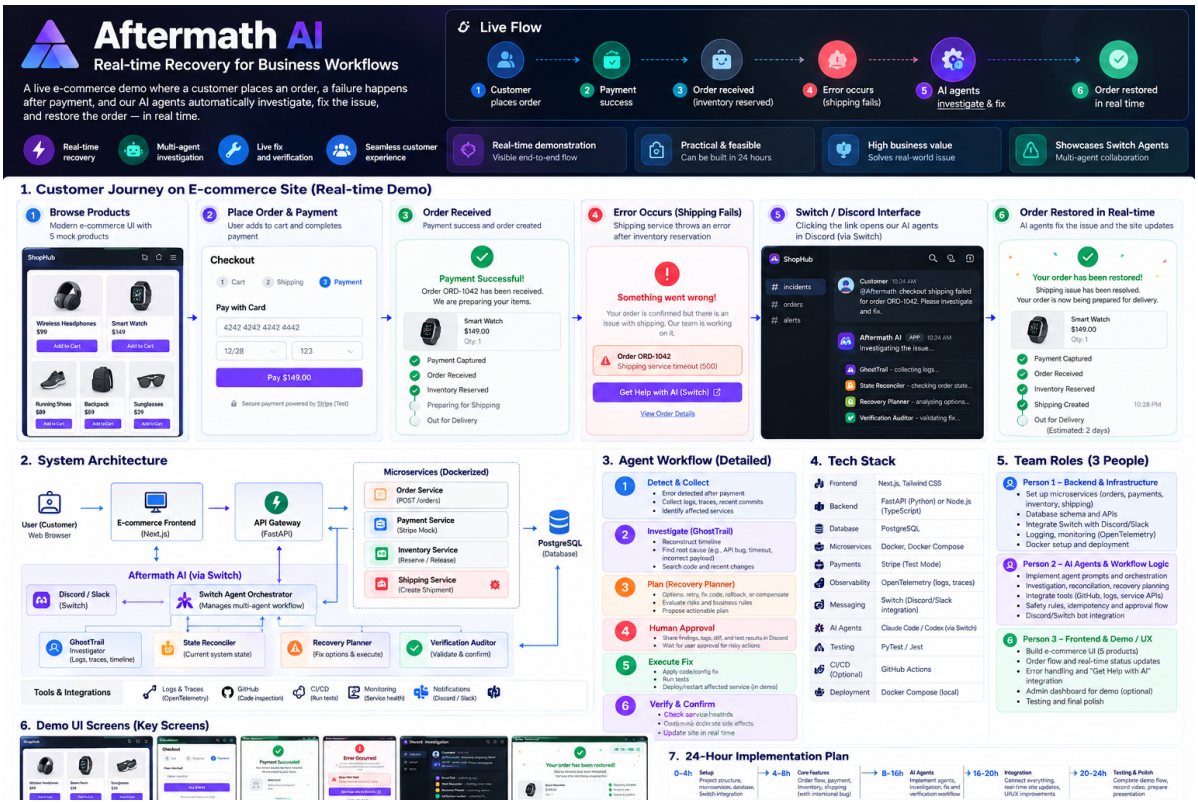
The verifier reads back order, payment, inventory, and shipment records. It checks that there is one order, no duplicate charge, inventory is consistent, and exactly one shipment exists.

12. Update the website

The backend emits an incident-status update through Server-Sent Events (SSE) or polling. The order page changes to “Order restored” only after verification. If recovery fails, it shows “We’re still resolving this” and a reference ID.

3. UI/UX Interface Concepts

Generated concept infographic



The infographic is a design direction. Implement the actual screens with real API-driven state and accessible interaction—not as a single static image.

Screens to build

1. Storefront: Five mock products, search/filter optional, product cards, cart, checkout CTA.

2. Checkout: Order summary, simulated payment method, clear “Demo payment—no real charge” disclosure.

3. Order received: Order ID, payment confirmed, inventory reserved, fulfillment pending.

4. Failure and recovery CTA: Honest message: payment/order confirmed; shipping needs attention. Button opens the configured Switch/Discord incident room.

5. Incident room: Agent statuses, incident timeline, log/code evidence, questions, proposed fix, approval control.

6. Recovery progress: Live phases: collecting evidence → investigating → reproducing → tests → awaiting approval → executing → verifying.

7. Restored order: Verified shipment/order state, final timeline, delivery estimate placeholder, incident reference, support/audit link.

8. Failure fallback: If unresolved, show “We’re still working on this” with incident ID; never falsely show restored.

4. System Architecture

Logical flow

Customer browser → Next.js storefront → backend/API → Order/Payment/Inventory/Shipping mock services → PostgreSQL event and incident store. On fault, backend creates an incident and sends a message through the Switch adapter to Discord. The Aftermath orchestrator coordinates investigator, repository inspector, reproducer/fixer, recovery planner, and verification auditor. Approved actions run only in a sandbox through a deterministic policy runner. Backend publishes status via SSE; browser refetches authoritative order state and displays restoration only after verification.

Layer	MVP choice	Purpose
Customer storefront	Next.js + TypeScript + Tailwind CSS	Responsive storefront, five mock products, cart, checkout, order status and recovery state.
API/backend	FastAPI (Python) OR Node.js/TypeScript; pick one	Order lifecycle, fault injection, incident creation, orchestration, SSE status events.
Business services	Mock Order, Payment, Inventory and Shipping APIs	Separate controllable services make partial failure and recovery visible.
Switch + chat	Switch + Discord (one platform for MVP)	Incident room, agent messages, evidence summaries, approval and final report.
Agent layer	Role-based agents via supported Switch protocol	Investigation, code analysis, test/reproduction, recovery planning and verification.
Safe execution	Docker sandbox + fixed action allowlist	Run tests and apply the demo fix without access to real production systems.
Persistence	PostgreSQL	Products, orders, inventory, shipments, events, incidents, approvals and audit trail.
Realtime updates	SSE first; polling fallback	Push incident progress and verified order status to the browser.
Testing	Pytest or Vitest/Jest + integration tests	Fault scenarios, idempotency, business invariants, API and UI flow.
Local environment	Docker Compose	Repeatable setup for app, database, mock services and agent adapter.

Recommended implementation choice

Use Next.js + TypeScript + Tailwind for the storefront. For the backend, choose either FastAPI (Python) if the team is faster in Python, or Node.js/TypeScript if the team wants one language end-to-end. Do not implement two competing backends. PostgreSQL stores authoritative state. Use SSE first for browser status updates, with polling as a fallback.

5. Agent Workflow and Guardrails

Agent/Service	Responsibility	Safety boundary
GhostTrail Investigator	Correlates event IDs, logs, timestamps and service responses into an evidence-linked incident timeline.	Cannot invent evidence; references must resolve to actual records.
Repository Inspector	Searches relevant code, Git diff/history, callers, configuration and tests for the failing path.	Read-only by default; repository content is untrusted input.
Reproducer & Fixer	Runs a minimal reproduction and regression test, proposes a small patch, then executes tests in an isolated workspace.	No production shell or unrestricted credentials; no merge/deploy without policy checks.
Recovery Planner	Chooses retry, compensate, repair, or escalate based on current state and business invariants.	Proposes actions only; cannot bypass deterministic policy.
Verification Auditor	Reads back final order/payment/inventory/shipment state and checks invariants.	Cannot mark success based only on model output or an HTTP 200 response.
Deterministic policy/runner	Validates permissions, approvals, allowlisted actions, idempotency keys, sandbox boundaries and execution results.	Only component allowed to execute an approved action.

Agent output contract

Every agent should return structured JSON with incident_id, findings, evidence_ids, uncertainty, proposed_next_step, side_effects, and approval_required. Validate evidence_ids against the database. The model can recommend a fix; it must not decide its own permissions or bypass policy checks.

Required business invariants

• Payment is simulated/test-mode only; no real payment credentials are collected or stored.

• A captured payment is never charged again during recovery.

• At most one shipment is created for a given order; retries reuse a stable idempotency key.

• A timeout is an unknown outcome until the shipping service is queried by order/idempotency key.

• Inventory reservation and release must agree with the verified order and shipment state.

• The UI says “Order restored” only after backend verification passes.

• If code tests fail, the sandbox is unavailable, or state remains ambiguous, the system must not claim success.

• Every proposal, approval, execution, and verification result is recorded in an audit trail.

6. Technology Stack

Area	Recommended stack	Reason
Frontend	Next.js, TypeScript, Tailwind CSS	Polished responsive shopping and order-status UX.
Backend	FastAPI OR Node.js/TypeScript	Order lifecycle, incidents, workflow state, SSE.
Database	PostgreSQL	Persistent orders, stock, shipments, incidents, events, approvals.
Switch/chat	Switch + Discord	Shared room, agent handoffs, approval and final report.
Agent tools	Supported Switch protocol; repository search; test runner	Evidence-based investigation and controlled tools.
Sandbox	Docker container with restricted permissions	Run reproduction/tests and apply demo patch safely.
Mock payment	Local simulated payment API	No real payment credentials or charges.
Testing	Pytest or Vitest/Jest + API integration tests	Regression, idempotency and invariant checks.
Realtime	Server-Sent Events + refetch fallback	Progress/status reaches the browser quickly.
Local dev	Docker Compose, GitHub Actions optional	Reproducible local setup and CI.

7. Database and Events

Entity	Example fields
products	product_id, name, price, stock, image_url
orders	order_id, customer_session_id, status, total, version
order_items	order_id, product_id, quantity, unit_price
payments	payment_id, order_id, status, simulated_provider_ref, idempotency_key
inventory_reservations	reservation_id, order_id, product_id, quantity, status
shipments	shipment_id, order_id, status, idempotency_key, provider_ref
events	event_id, incident_id, order_id, source, type, occurred_at, received_at, correlation_id, payload
incidents	incident_id, order_id, status, severity, created_at, resolution_summary
recovery_plans/actions	plan_id, steps, preconditions, approval_status, action_id, idempotency_key, result
audit_log	actor, action, timestamp, reason, evidence references

8. Three-Person Work Distribution

Person	Ownership	Deliverables
Person 1 — Backend, data & Switch integration	Order lifecycle APIs, mock services, DB schema, fault injection, Switch/Discord adapter, SSE status updates.	A working customer-order flow; incident ID/correlation ID; persisted logs/events; chat trigger; browser status updates.
Person 2 — Agents, code inspection & safe recovery	Agent roles/tools, log/code inspection, reproduction, regression tests, recovery planner, approval and policy checks.	Evidence-linked timeline; validated agent JSON; sandboxed test/fix path; safe recovery plan; verifier and invariants.
Person 3 — Frontend, UX & QA/demo	Storefront, five product cards, cart/checkout, order received/error/restored screens, incident progress view, test plan and demo.	Polished responsive UI; recovery CTA; real-time status; accessible loading/error states; seeded demo; README and demo recording.

Integration rule: Person 1 should expose stable API contracts early; Person 2 should consume those APIs and return validated JSON; Person 3 should wire screens to the same backend status fields. Agree on status enums before parallel work.

9. 24-Hour Implementation Plan

Time	Build focus	Done when
0–3 h	Lock scenario, contracts, schema, UI states, Switch integration approach.	One end-to-end sequence agreed; a simple Switch message round trip works.
3–7 h	Build five-product storefront, cart, order API, mock payment/inventory/shipping services.	Customer can order; seeded shipping fault appears after order/payment/inventory succeed.
7–11 h	Persist events/logs and incident state; implement SSE/polling updates.	Order page displays incident status from backend, not hard-coded frontend state.
11–15 h	Connect Switch/Discord and agent roles; implement evidence timeline and repo inspection.	A real incident message triggers agents and returns a structured investigation.
15–19 h	Sandboxed reproduction/fix path, regression tests, approval, idempotent runner.	Demo fix passes tests; risky actions require approval; repeat request does not duplicate shipment.
19–22 h	Read-back verification, order restored UI, failure/unknown fallback states.	Site updates only after invariants pass; unresolved scenarios show honest status.
22–24 h	UX polish, reset script, regression tests, README, demo recording and pitch.	One-click/resettable five-minute demo with a recorded fallback.

Cut if time is tight

Keep five products, one seeded bug, one chat platform, one backend, one successful recovery path, and one failure fallback. Defer real payment, actual production deployment, multiple chat platforms, sophisticated tracing, and automatic unrestricted code changes.

10. Five-Minute Demo Script

0:00–0:40

Browse five mock products, add one to cart, and place an order with a clearly labeled simulated payment.

0:40–1:10

Show payment captured, order received, and inventory reserved.

1:10–1:30

Trigger the seeded shipping bug. The site shows an honest error and an incident reference, not a false success.

1:30–2:20

Click the recovery CTA to open the configured Switch/Discord incident room. Show agents gathering logs, checking service state, inspecting the relevant code, and reproducing the failure.

2:20–3:20

Show a proposed patch and regression test. A designated operator approves the action; the sandbox runner executes it.

3:20–4:20

Show tests pass, shipping operation retried with an idempotency key, and read-back verification across order/payment/inventory/shipping.

4:20–5:00

Return to the order page. SSE updates the status to “Order restored” only after backend verification. Replay the action and show no duplicate shipment.

11. Success Metrics

• Recovery completion rate across seeded recoverable scenarios.

• Zero duplicate shipments and zero duplicate charges across replay tests.

• Time from injected fault to verified restoration.

• Percentage of findings linked to valid evidence records.

• Correct escalation rate for ambiguous or non-recoverable scenarios.

• Regression test pass rate and invariant violations (target: zero violations).

12. Risks and Mitigations

Switch integration mismatch	**Prove the smallest supported flow early: site/backend creates incident → Switch room receives it → agent response returns with incident ID.**
Unsafe AI code execution	Use a Docker sandbox with no production credentials, restricted network, timeouts, and a fixed command allowlist.
LLM hallucinated logs or code facts	Validate every evidence ID and quote against stored records; agent outputs are structured and checked.
Payment/shipping timeout ambiguity	Query current state before retrying; stable idempotency keys; escalate if state cannot be confirmed.
Realtime UI gets out of sync	Backend is the source of truth; SSE carries status events; browser can refetch authoritative order state.
24-hour scope too large	Use one seeded bug and mock services. Keep actual live payment, real production deployment, multi-platform support and advanced tracing out of scope.

13. Security and Safety

• Use mock products, simulated payment, and synthetic customer data only.

• Do not send secrets, environment variables, payment credentials, or unrestricted source dumps to agents.

• Treat repository files, logs, user messages, and external payloads as untrusted data.

• Run tests and patches in a disposable sandbox; block production credentials and unrestricted network access.

• Restrict Discord/Switch approval commands to authorized demo users.

• Use allowlisted operations, explicit approvals, stable idempotency keys, and audit logs.

• Backend owns the authoritative order status; UI cannot mark an order restored on its own.

14. Acceptance Checklist

■ Five products load from the backend/database and can be ordered.

■ Payment is explicitly simulated; no real charge or card details are collected.

■ The order is recorded and inventory reserved before the seeded shipping failure.

■ The failure creates a persistent incident_id and correlation_id.

■ The recovery CTA opens the configured Switch/Discord incident room.

■ Agents use real mock-service logs and code artifacts; every evidence reference is validated.

■ Code execution and patch tests occur in a restricted sandbox.

■ Consequential recovery actions require explicit operator approval.

■ Retries use idempotency and cannot create duplicate charges or shipments.

■ Backend verification controls the final order status; UI cannot self-declare success.

■ SSE updates the browser, and refresh still retrieves the authoritative status.

■ If recovery cannot be verified, the site shows a pending/needs-support state rather than “restored”.

■ README includes setup, environment variables, reset/seed script, architecture, tests, and demo instructions.

Project pitch

“Aftermath AI is a real-time e-commerce recovery demo. When a post-payment failure interrupts an order, specialized agents coordinate through Switch to investigate logs and code, reproduce the bug, propose and test a fix, and execute only approved actions in a sandbox. The backend verifies payment, inventory, and shipment state, then updates the customer’s order page in real time—without blind retries or duplicate side effects.”

Feasibility verdict: Strong hackathon idea if scoped to mock services, one seeded failure, a controlled sandbox, and a verified end-to-end recovery. The hardest part is safe, reliable code execution plus Switch integration; validate those early. Do not promise arbitrary production auto-repair in the MVP.